
LLD – DAY 6

SOLID Design Principles (Continued)

Today Covered

- LSP (in depth – all rules)
- ISP (4th principle)
- DIP (5th principle)

L – Liskov Substitution Principle (LSP) – Deep Dive

Core Definition (Revision)

Subclasses should be **substitutable** for their parent classes bina client code break kiye.

Agar:

```
Parent* p = new Child();
```

toh:

```
p->method();
```

har jagah same behavior dena chahiye.

Important Terms (Page 1)

 Broad

Parent / Ancestor class

Example:

```
Animal
```

 Narrow

Child class

Example:

Dog

Animal



Dog

◇ LSP Guidelines / Rules

LSP mainly **3 rules** follow karta hai:

1 Signature Rule 2 Property Rule 3 Method Rule

◇ 1. Signature Rule (Method Argument Rule)

☑ Rule

Agar parent class ke method me **arguments hain**, toh child class me:

- **same arguments**
- ya **broader arguments** allowed **✗ narrow arguments NOT allowed**

🗣 Hinglish Explanation

Client ko sirf parent ka pata hota hai. Agar child alag parameters expect karega, toh **runtime pe code fail ho jayega**.

✍ Text Diagram

```

Client
|
v
Parent
solve(string s)

Child
solve(string s)    ✓ allowed
solve(int x)       ✗ NOT allowed
  
```

🔗 **Reason** Client string bhej raha hai → child int expect kare → crash **✗**

◇ Return Type Rule (Part of Signature Rule)

☑ Rule

Child class ka return type:

- **same**
- ya **narrower (covariant)** ho sakta hai ✗ broader return type NOT allowed

🧠 Example

Hierarchy:

Organism



Animal



Dog

✍ Text Diagram

Parent:

Animal random()

Child:

Dog random() ✓ OK (covariance)

Organism random() ✗ NOT allowed

🔗 **Covariance** = returning more specific type

◇ Exception Rule

☑ Rule

Agar parent method exception throw karta hai:

- Child **same**
- ya **narrow exception** throw kar sakta hai ✗ broader exception NOT allowed

✍ Exception Hierarchy

```

logic_error
├── invalid_argument
├── domain_error
└── out_of_range

```

Text Diagram

```

Parent:
throws logic_error

Child:
throws invalid_argument ✓ allowed
throws runtime_error ✗ NOT allowed

```

 **Reason** Client sirf `logic_error` handle karta hai Broader error aaya → crash ✗

◇ 2. Property Rule

◇ Invariant Rule

☒ Definition

Jo condition **parent class me hamesha true** hai wahi condition **child class me bhi true rehni chahiye**

 Example (Account)

```

Invariant:
balance >= 0

```

Text Diagram

```

Account
balance >= 0    ✓ invariant

CheatAccount
balance = -1    ✗ invariant broken

```

 **Result** CheatAccount **parent ka substitute nahi ban sakta** → LSP break ✗

◇ History Constraint Rule

☑ Rule

Child class **past behavior change nahi kar sakta**

🧠 Example

```
Account  
withdraw() allowed  
  
FixedAccount  
withdraw() -> throws exception ✕
```

✍ Text Diagram

```
Account  
withdraw()  
  
FixedAccount  
withdraw() -> exception ✕
```

✂ Withdraw pehle allowed tha, ab nahi → **LSP break**

◇ Immutable Class / Method

◇ Immutable Class

Jise inherit ya change nahi kar sakte (C++: class ko **final** bana do)

◇ Immutable Method

Jise override nahi kar sakte (C++: method ko **final** bana do)

✂ Agar parent immutable hai aur child mutable bana diya → **History constraint break**

◇ 3. Method Rule

◇ a. Pre-condition Rule

☑ Rule

Child class **pre-condition strengthen nahi kar sakta**

Example

Parent pre:
num > 0 && num <= 5

Child pre:
num > 0 && num <= 10 ✓ OK (weaker)
num > 0 && num <= 3 ✗ NOT OK (stronger)

Text Diagram

Parent m1(int num)
Pre: 0 < num <= 5

Child m1(int num)
Pre: 0 < num <= 10 ✓

 Client jo parent ke hisaab se valid input de raha hai child usse reject nahi kar sakta

◇ Real Life Example

User
createPassword()

AdminUser
createPassword()

Parent pre: password >= 6
Child pre: password >= 6 ✓

✓ Safe substitution

◇ b. Post-condition Rule

☒ Rule

Child class **post-condition weaken nahi kar sakta** Strengthen kar sakta hai ✓

Example (Car)

```
Car  
brake() -> speed slow down  
  
ElectricCar  
brake() -> speed slow + charging ✓
```

Text Diagram

```
Car  
brake()  
  
ElectricCar  
brake()  
+ chargeBattery()
```

 Extra behavior allowed → LSP safe ✓

◇ I – Interface Segregation Principle (ISP)

☑ Definition

Many **client-specific interfaces** are better than **one general-purpose interface**

 Clients ko **unwanted methods force nahi karne chahiye**

◇ Problem Example

```
<<abstract>>  
Shape  
area()  
volume()
```

```
Square  
Rectangle  
Cube
```

✗ Square / Rectangle ko volume nahi chahiye Phir bhi implement karna pad raha hai → ISP break

Text Diagram (Violation)

```
Shape
area()
volume()
```

```
Square -> volume() ✗
Rectangle -> volume() ✗
```

◇ Solution (ISP Applied)

Split interfaces 

Text Diagram (Correct)

```
<<abstract>> 2DShape
area()
```

```
<<abstract>> 3DShape
area()
volume()
```

```
Square -> 2DShape
Rectangle -> 2DShape
Cube -> 3DShape
```

 No unwanted methods ✓  ISP satisfied ✓

◇ D – Dependency Inversion Principle (DIP)

☑ Definition

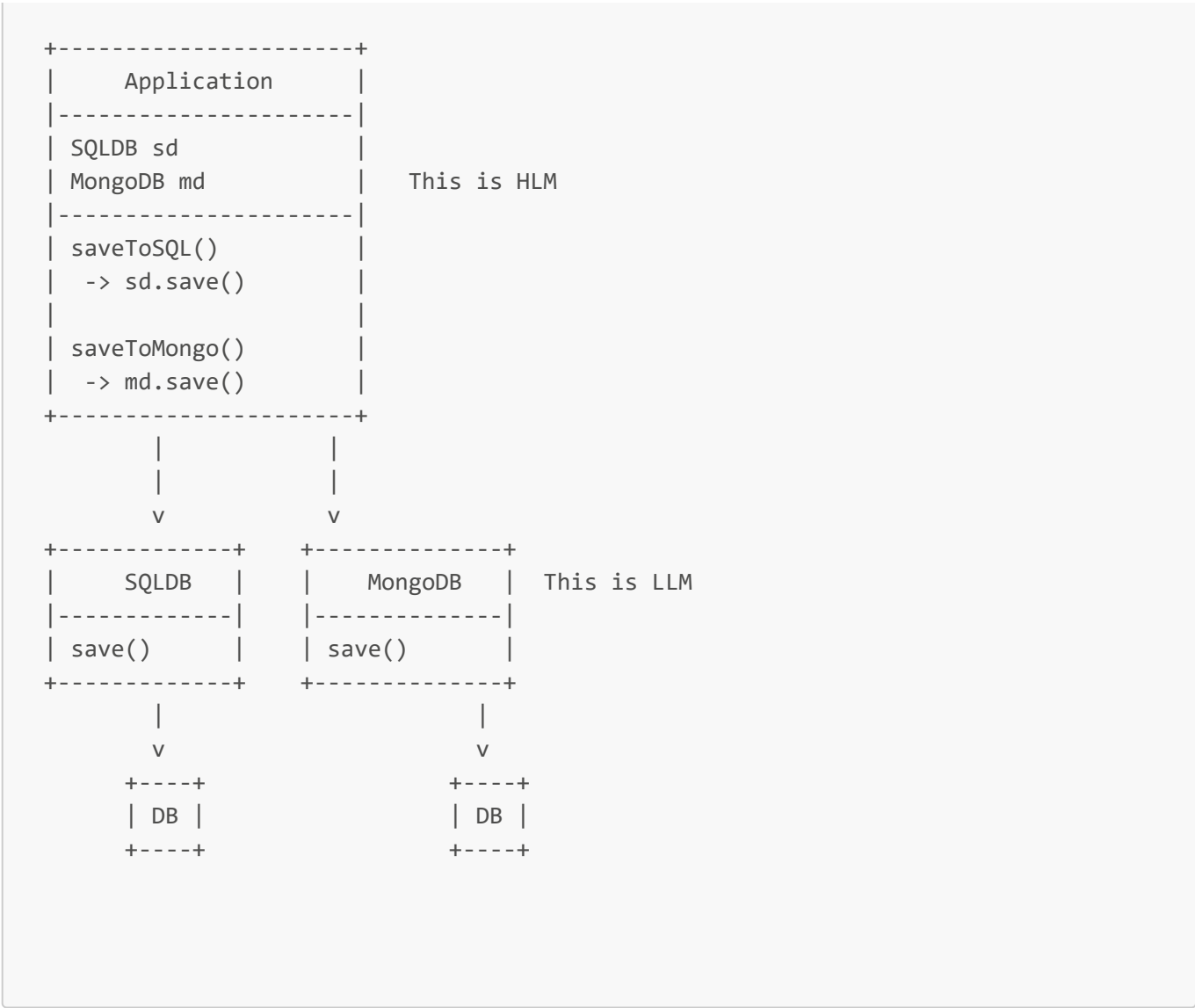
High Level Module (HLM) should NOT depend on Low Level Module (LLM) Both should depend on **abstraction**

◇ HLM vs LLM

- **HLM** → Business logic
- **LLM** → DB, File, External API

◇ Problem (Without DIP)

Text Diagram (Violation)

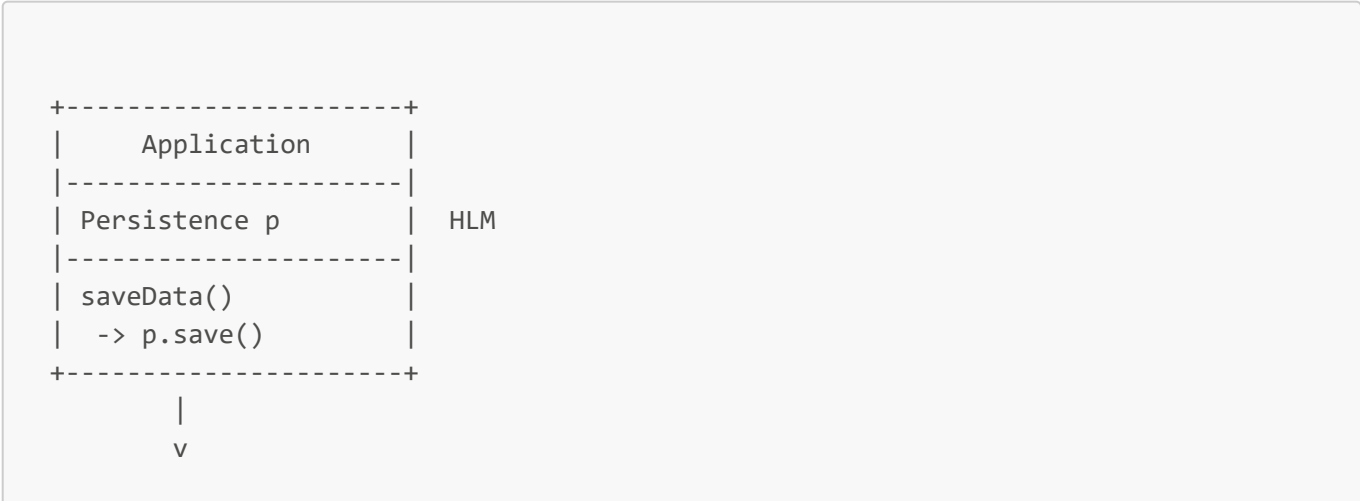


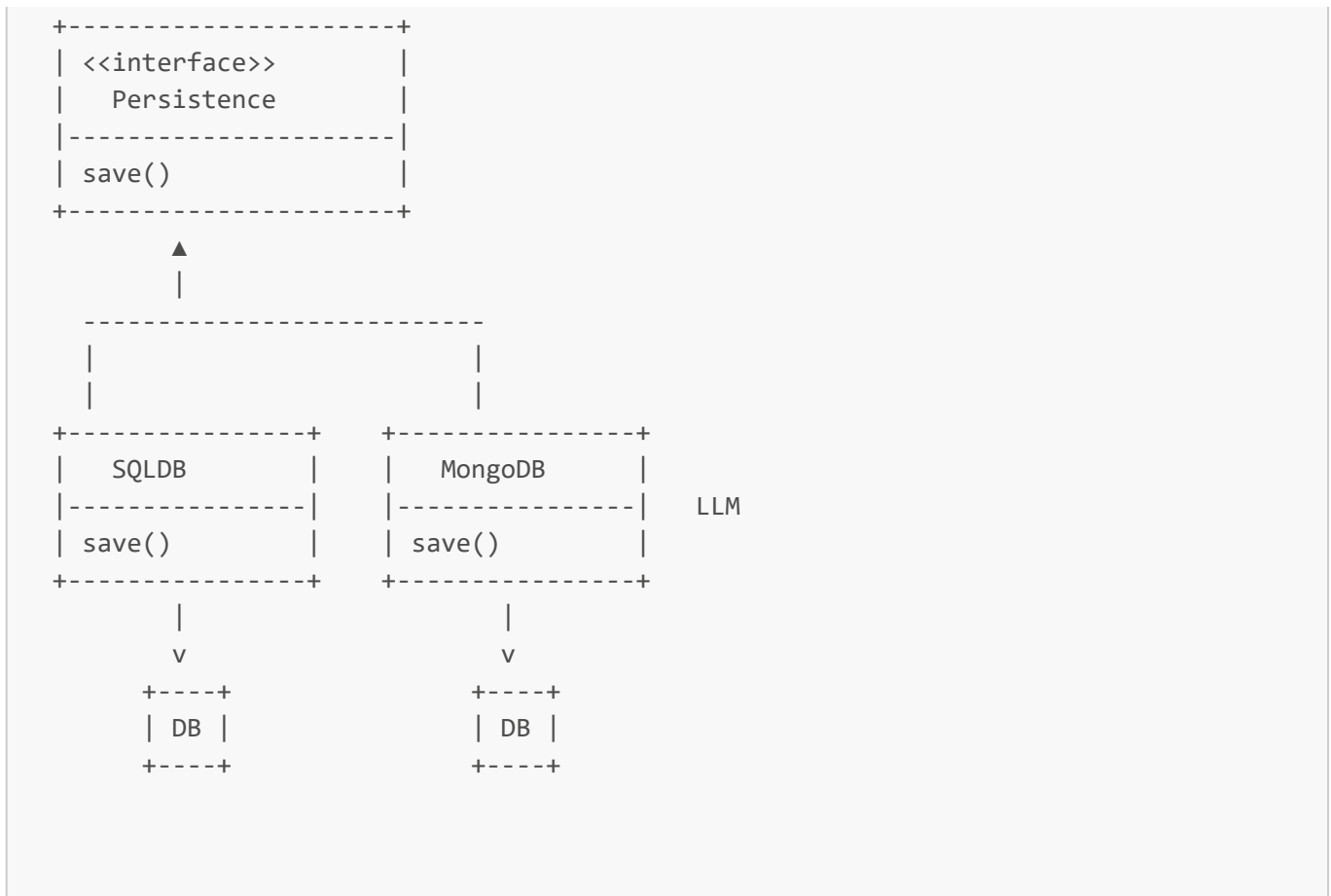
✂ Agar MongoDB → Redis change kiya 🔄 Application code change karna padega 🔄 OCP + DIP break ❌

◇ **Solution (DIP Applied)**

Use **interface / abstraction**

✎ **Text Diagram**





🧠 Explanation

Ab DB change karne pe:

- Application code same rahega
- Sirf new implementation add hogi

✓ DIP satisfied ✓ OCP satisfied

📄 DAY 6 SUMMARY

✓ LSP = safe substitution ✓ Signature, Property, Method rules ✓ ISP = no forced methods ✓ DIP = depend on abstraction ✓ All principles reduce coupling

🔗 Interview Golden Lines

- LSP ensures polymorphism correctness
- Pre-condition weak, Post-condition strong
- ISP avoids fat interfaces
- DIP + OCP go hand in hand

🧠 Conclusion: SOLID Design Principles (In Short)

S – Single Responsibility Principle (SRP)

☞ *Ek class = ek kaam* ☞ Agar ek class ke **multiple reasons to change** hain, to SRP break hota hai.

🗣️ *"One class, one responsibility"*

O – Open Closed Principle (OCP)

☞ Code ko **extend kar sakte ho**, par **modify nahi karna chahiye** ☞ New feature aaye → **new class add karo**, purani class mat chhedo.

🗣️ *"Extend, don't modify"*

L – Liskov Substitution Principle (LSP)

☞ Child object ko parent ki jagah use karo → **code break nahi hona chahiye** ☞ Child ko parent jaisa hi behave karna chahiye.

🗣️ *"Child must behave like parent"*

I – Interface Segregation Principle (ISP)

☞ Client ko **unwanted methods force mat karo** ☞ **Small, specific interfaces** better hote hain than one big interface.

🗣️ *"Many small interfaces are better than one fat interface"*

D – Dependency Inversion Principle (DIP)

☞ High-level code ko **low-level code pe depend nahi karna chahiye** ☞ Dono ko **abstraction (interface)** pe depend karna chahiye.

🗣️ *"Depend on abstraction, not on implementation"*